Chapitre 1

Partie 2- Gestion de l'intégrité - Corrigé des séries TD et TP

BELKASMI Djamal

20/10/2024

3ème Année Ingénieurs & Master 1

Corrigé de la série d'exercices N°1

Exercice 1 :

Code SQL

```sql
CREATE TABLE employes (
    id_employe INT PRIMARY KEY,
    nom VARCHAR(50),
    prenom VARCHAR(50),
    email VARCHAR(100)
) ENGINE=InnoDB;
```

Exercice 2 :

Code SQL

```sql
CREATE TABLE departements (
    id_departement INT PRIMARY KEY,
    nom_departement VARCHAR(50),
) ENGINE=InnoDB;

CREATE TABLE employes (
    id_employe INT PRIMARY KEY,
    nom VARCHAR(50),
    prenom VARCHAR(50),
    email VARCHAR(100),
    id_departement INT,
    FOREIGN KEY (id_departement) REFERENCES departements(id_departement)
) ENGINE=InnoDB;
```

Exercice 3 :

Code SQL

```sql
CREATE TABLE Produits (
    ID_Produit INT PRIMARY KEY,
    Nom VARCHAR(50),
    Prix DECIMAL(10, 2),
    CONSTRAINT chk_Prix CHECK (Prix > 0)
);
```

Exercice 4 :

Code SQL

```sql
CREATE TABLE Utilisateurs (
    ID_Utilisateur INT PRIMARY KEY,
    Nom VARCHAR(50),
    Email VARCHAR(100) UNIQUE
);
```

Exercice 5 :

Code SQL

```sql
CREATE TABLE Commandes (
    ID_Commande INT PRIMARY KEY,
    Date_Commande DATE NOT NULL,
    Montant DECIMAL(10, 2) NOT NULL
);
```

Exercice 6 :

Code SQL

```sql
CREATE TABLE Inscriptions (
    ID_Etudiant INT,
    ID_Cours INT,
    Date_Inscription DATE,
    PRIMARY KEY (ID_Etudiant, ID_Cours)
);
```

Exercice 7 :

Code SQL

```sql
CREATE TABLE Salaires (
    ID_Employé INT PRIMARY KEY,
    Nom VARCHAR(50),
    Salaire DECIMAL(10, 2),
    CONSTRAINT chk_Salaire CHECK (Salaire >= 1500)
);
```

Exercice 8 :

Code SQL

```sql
CREATE TABLE projets (
   id_projets INT PRIMARY KEY,
   nom_projets VARCHAR(50)
) ENGINE=InnoDB;

CREATE TABLE taches (
   id_taches INT PRIMARY KEY,
   description VARCHAR(100),
   id_projets INT,
   FOREIGN KEY (id_projets) REFERENCES projets(id_projets) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Exercice 9 :

Code SQL

```sql
CREATE TABLE Reservations (
    ID_Reservation INT PRIMARY KEY,
    ID_Utilisateur INT,
    ID_Chambre INT,
    Date_Reservation DATE,
    UNIQUE (ID_Utilisateur, ID_Chambre, Date_Reservation)
);
```

Exercice 10 :

Code SQL

```sql
CREATE TABLE Produits (
    ID_Produit INT PRIMARY KEY,
    Nom VARCHAR(50),
    Prix DECIMAL(10, 2),
    EnPromotion BOOLEAN,
    CONSTRAINT chk_Promotion CHECK (EnPromotion = FALSE OR Prix < 20)
);
```

Corrigé de la série d'exercices N°2

Exercice 1 :

Code SQL

```sql
CREATE TABLE Produits (
    id INT PRIMARY KEY,
    nom VARCHAR(100),
    prix DECIMAL(10, 2),
    quantite INT,
    CHECK (prix > 0)  -- Le prix doit être supérieur à 0
);
```

Exercice 2 :

Code SQL

```sql
CREATE TABLE departements (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL
) ENGINE = InnoDB;

CREATE TABLE employes (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL,
    departement_id INT,
    FOREIGN KEY (departement_id) REFERENCES departements(id)
) ENGINE = InnoDB;
```

Exercice 3:

Code SQL

```sql
CREATE TABLE Clients (
    id INT PRIMARY KEY,
    nom VARCHAR(50),
    age INT
);

CREATE TRIGGER avant_insertion_client
BEFORE INSERT ON Clients
FOR EACH ROW
BEGIN
    IF NEW.age < 18 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Le client doit avoir au moins 18 ans.';
    END IF;
END;
```

Exercice 4 :

Code SQL

```sql
CREATE TABLE Ventes (
    id INT PRIMARY KEY,
    id_produit INT,
    quantite_vendue INT
);

CREATE TRIGGER apres_vente
AFTER INSERT ON Ventes
FOR EACH ROW
BEGIN
    UPDATE Produits
    SET quantite = quantite - NEW.quantite_vendue
    WHERE id = NEW.id_produit;
END;
```

Exercice 5 :

Code SQL

```sql
CREATE TABLE Produits (
    id INT PRIMARY KEY,
    nom VARCHAR(100),
    etat VARCHAR(20),
    CHECK (etat IN ('neuf', 'occasion', 'défectueux'))  -- L'état doit être l'un des trois
);
```

Exercice 6 :

Code SQL

```sql
CREATE TABLE Commandes (
    id INT PRIMARY KEY,
    date_commande DATE,
    montant DECIMAL(10, 2),
    CHECK (date_commande <= CURDATE())  -- La date de commande ne peut pas être dans le futur
);
```

Exercice 7 :

Code SQL

```sql
CREATE TABLE Produits (
    id INT PRIMARY KEY,
    nom VARCHAR(100),
    quantite INT
);

CREATE TRIGGER avant_suppression_produit
BEFORE DELETE ON Produits
FOR EACH ROW
BEGIN
    IF OLD.quantite > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Impossible de supprimer un produit avec une quantité supérieure à 0.';
    END IF;
END;
```

Exercice 8 :

Code SQL

```sql
CREATE TABLE HistoriqueVentes (
    id INT PRIMARY KEY AUTO_INCREMENT,
    id_produit INT,
    quantite INT,
    date_vente DATETIME
);

CREATE TRIGGER apres_vente
AFTER INSERT ON Ventes
FOR EACH ROW
BEGIN
    INSERT INTO HistoriqueVentes (id_produit, quantite, date_vente)
    VALUES (NEW.id_produit, NEW.quantite_vendue, NOW());
END;
```

TP : 01
Gestion des Contraintes d'Intégrité dans une Base de Données

1. Création de la Base de Données

Code SQL

```sql
CREATE DATABASE entreprise;
USE entreprise;
```

2. Création des Tables

Code SQL

```sql
CREATE TABLE departements (
   id INT PRIMARY KEY AUTO_INCREMENT,
   nom VARCHAR(100) UNIQUE NOT NULL
) ENGINE = InnoDB;
```

Code SQL

```sql
CREATE TABLE employes (
   id INT PRIMARY KEY AUTO_INCREMENT,
   nom VARCHAR(100) NOT NULL,
   salaire DECIMAL CHECK (salaire > 1500),
   departement_id INT,
   FOREIGN KEY (departement_id) REFERENCES departements(id)
) ENGINE = InnoDB;
```

3. Insertion de Données

Code SQL

```sql
-- Insertion valide
INSERT INTO departements (nom) VALUES ('Ressources Humaines'), ('Informatique'), ('Marketing');
```

Code SQL

```sql
-- Insertion valide
INSERT INTO employes (nom, salaire, departement_id) VALUES ('Omar', 2000, 1);
INSERT INTO employes (nom, salaire, departement_id) VALUES ('Ahmed', 1600, 2);
```

Remarque: L'exécution du code d'insertion ci-dessus s'exécute correctement vu que les contraintes sont vérifiées.

Code SQL

```sql
-- Insertion non valide
INSERT INTO employes (nom, salaire, departement_id) VALUES ('Khaled', 1200, 3);
```

Remarque: L'exécution du code d'insertion ci-dessus s'exécute avec une erreur vu que la contrainte **CHECK** sur l'attribut *'salaire'* n'est pas vérifiée (salaire >=1500).

4. Mise à Jour des Données

Code SQL

```sql
-- Mise à jour valide
UPDATE employes SET salaire = 2500 WHERE nom = 'Ahmed';
```

Remarque: L'exécution du code de mise à jour ci-dessus s'exécute correctement vu que les contraintes sont vérifiées.

Code SQL

```sql
-- Mise à jour invalide (devrait échouer)
UPDATE employes SET salaire = 1000 WHERE nom = 'Omar';
```

Remarque: L'exécution du code de mise à jour ci-dessus s'exécute avec une erreur vu que la contrainte **CHECK** sur l'attribut *'salaire'* n'est pas vérifiée (salaire >=1500).

5. Suppression de Données

Code SQL

```sql
-- Suppression invalide (devrait échouer)
DELETE FROM departements WHERE id = 1;
```

Remarque: L'exécution du code de suppression ci-dessus s'exécute avec une erreur vu que la contrainte **FOREIGN KEY** n'est pas vérifiée (le département n°1 est lié à des employés).

Code SQL

```sql
-- Suppression valide
DELETE FROM employes WHERE nom = 'Omar';
```

Remarque: L'exécution du code de suppression ci-dessus s'exécute correctement vu qu'aucune contrainte n'est mise en cause.

TP 02
Gestion des Contraintes d'Intégrité Statiques et Dynamiques

1. Création de la Base de Données

Code SQL

```sql
CREATE DATABASE gestion_projets;
USE gestion_projets;
```

2. Création des Tables avec Contraintes Statiques

Code SQL

```sql
CREATE TABLE projets (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(100) UNIQUE NOT NULL,
    budget DECIMAL CHECK (budget > 0)
)ENGINE = InnoDB;
```

Code SQL

```sql
CREATE TABLE employes (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL,
    role VARCHAR(50) NOT NULL,
    salaire DECIMAL CHECK (salaire > 1500)
)ENGINE = InnoDB;
```

Code SQL

```sql
CREATE TABLE affectations (
    id INT PRIMARY KEY AUTO_INCREMENT,
    projet_id INT,
    employe_id INT,
    date_debut DATE NOT NULL,
    date_fin DATE NOT NULL,
    FOREIGN KEY (projet_id) REFERENCES projets(id),
    FOREIGN KEY (employe_id) REFERENCES employes(id),
```

```
  CHECK (date_fin > date_debut)
)ENGINE = InnoDB;
```

3. Insertion de Données

Code SQL

```
-- Insertion valide

INSERT INTO projets (nom, budget) VALUES ('Projet A', 50000);
INSERT INTO projets (nom, budget) VALUES ('Projet B', 30000);

INSERT INTO employes (nom, role, salaire) VALUES ('Ali', 'Développeur', 2500);
INSERT INTO employes (nom, role, salaire) VALUES ('Brahim', 'Chef de Projet', 3000);
```

Remarque: L'exécution du code d'insertion ci-dessus s'exécute correctement vu que les contraintes sont vérifiées.

Code SQL

```
-- Insertion non valide

INSERT INTO projets (nom, budget) VALUES ('Projet A', 70000); -- Nom déjà utilisé
INSERT INTO employes (nom, role, salaire) VALUES ('Chanez', 'Analyste', 1200); -- Salaire invalide
```

Remarque: L'exécution du code d'insertion ci-dessus s'exécute avec des erreurs vu que les contraintes ne sont pas vérifiées.

4. Création de Déclencheurs (Triggers)

Code SQL

```
CREATE TRIGGER check_budget
BEFORE INSERT OR UPDATE ON projets
FOR EACH ROW
BEGIN
  IF NEW.budget > 100000 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Le budget ne peut pas dépasser 100 000 DA.';
  END IF;
END;
```

5. Test des Déclencheurs

Code SQL

```sql
-- Insertion valide

INSERT INTO projets (nom, budget) VALUES ('Projet C', 80000);
```

Remarque: à l'exécution du code d'insertion ci-dessus le code du déclencheur **check_budget** s'exécute automatiquement et ne renvoie aucune erreur.

Code SQL

```sql
-- Insertion non valide

INSERT INTO projets (nom, budget) VALUES ('Projet D', 150000); -- Budget > 100000 DA
```

Remarque: à l'exécution du code d'insertion ci-dessus le code du déclencheur **check_budget** s'exécute automatiquement et en testant la valeur du budget il renvoie un message d'erreur signalant que le montant est supérieur à 100000 DA.

6. Mise à Jour des Données

Code SQL

```sql
-- Insertion valide

UPDATE employes SET salaire = 3500 WHERE nom = 'Ali';
```

Remarque: à l'exécution du code d'insertion ci-dessus la contrainte **CHECK** ne renvoie aucune erreur.

Code SQL

```sql
-- Insertion non valide (date_fin < date_debut)

UPDATE affectations SET date_fin = '2023-01-01', date_debut = '2023-01-05' WHERE id = 1;
```

Remarque: à l'exécution du code d'insertion ci-dessus la contrainte CHECK renvoie une erreur indiquant que date_fin < date_debut

7. Suppression de Données

Code SQL

```
-- Suppression invalide (devrait échouer)
DELETE FROM projets WHERE id = 1;  -- projet affecté
```

Remarque: L'exécution du code de suppression ci-dessus s'exécute avec une erreur vu que la contrainte **FOREIGN KEY** n'est pas vérifiée (le projet n°1 est lié à des employés dans la table affectation).

Code SQL

```
-- Suppression valide
DELETE FROM employes WHERE nom = 'Brahim';
```

Remarque: L'exécution du code de suppression ci-dessus s'exécute correctement vu qu'aucune contrainte n'est mise en cause.